

CopyCat

CISS 346 – M5 Project Report

Team Members

Amanda Bradburn

William Davis

Nolan Biggers



Overview and Initial Design

Those just beginning to dip their toes into the world of programming often struggle to properly secure their code, leading to potential threats if code is exposed or publicly accessible. Learning how to properly secure your code can be a hard task too, leading many to improperly manage it. In order to hopefully help these beginners to find and fix the weaknesses and vulnerabilities in their code, our group decided to build CopyCat, a tool to scan code, primarily Python, and report vulnerabilities back to the user in a easily digestible, understandable format.

Our primary goal when beginning this project was to create a tool that, when given a snippet of Python code, would provide the user with an easily understandable breakdown of any issues found. We planned to use AI to provide the explanations, as well as personalized fixes that specifically tell the user what to do to fix their code. While our initial plans also involved only having this tool run locally, we eventually shifted to a version running on a live website in order to get around some issues with API keys.

The entire system can be broken down into 3 main parts:

- Frontend:
 - Handles all the user-facing parts of the program, including the website for submission and viewing results
 - Uses JavaScript to send code and receive/format results
- Backend:
 - Flask API to tie everything together
- Analysis Pipeline:
 - Sends user-submitted code to Bandit, then sends Bandit report and code sample to Gemini API for final result

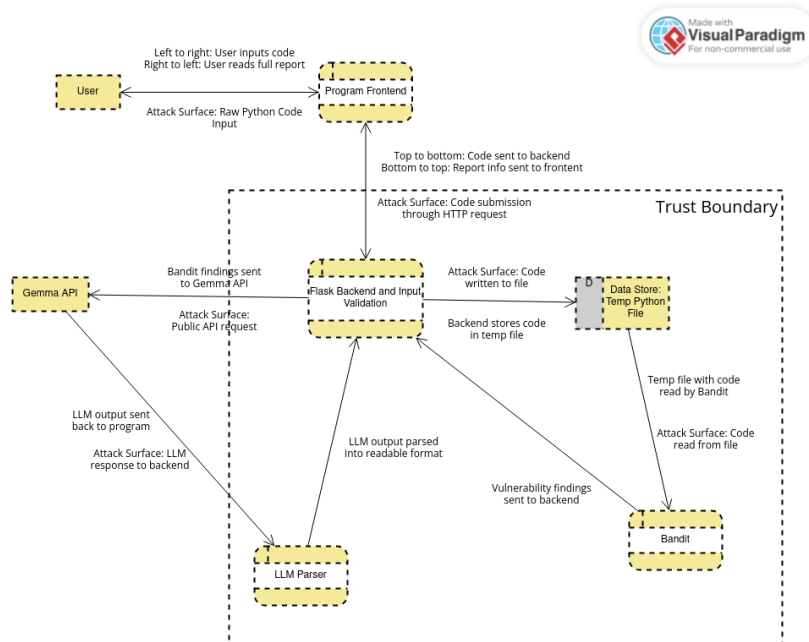
Development Lifecycle and Methodology

For this project, we followed a more agile-focused developmental cycle, working in individual features and slowly putting together a complete package. Our first iteration was extremely simple; just enough was put together in order to have a working proof of concept. The website had minimal CSS, was yet to be put on a live website, and many bugs were present. Throughout the next several weeks, we improved the project bit by bit. The website CSS and result output were cleaned up, making everything look more complete. Support for more languages was added (with full AI analysis), along with a secure method to allow users access to more powerful, costly AI models. We also put the program on a live website after working out it would be the easiest solution to deal with API keys, as we had no easy way to distribute them.

Many of our specific tools were chosen early on and stuck with us throughout the project. Python was chosen because it was a standard and easy to use programming language, and Bandit was used due to its easy implementation with our Python program.

Gemini was chosen as our AI due to a variety of powerful, fast, and cheap models giving a wide spread of options for which to use. We also had \$10 of free credit on the Gemini API, which would hopefully keep costs down. By default, the program uses Gemini 2.5 Flash (pretty cheap and fast, but good enough when combined with grounding from the Bandit report). The user has an option to enter a passcode when submitting a request from the website, and if the passcode matches what's in the .env file, the model changes appropriately. We took advantage of this to add support for languages fully using AI for analysis as well, with the options for more powerful models increasing output quality. The API key was securely stored inside the .env file alongside these passcodes, was not pushed to GitHub, and was shared to project members via alternate methods.

Threat Modeling and Security Analysis



After completing our data flow diagram and threat modeling, several main attack surfaces were identified, mainly involving the usage of Gemini. Our primary concern was the potential attempts to use prompt injection to manipulate the output of the AI, creating false reports or unintended output. We were also worried about the AI randomly not sticking to its instructions and returning output in a format that wouldn't work in the frontend. In order to mitigate the risks of both, an extensive prompt was put together for the AI, including instructions to not analyze the code itself and blocks around the user's code preventing it from being mistakenly seen as part of the prompt. Other steps were taken to do some basic input validation (stopping spam/oversized requests), and the usage of a .env file to protect the Gemini API key and any implemented passcodes. Basic logging was also added to the Python program.

Testing Results and Reliability

Throughout the variety of test cases used, the program returned output as expected. Output was never observed to be returned in a bad format either, sticking to the specified JSON format specified in the prompt. Due to the usage of Bandit, the program correctly caught the majority of vulnerabilities consistently. In the example below, Bandit correctly identified the usage of `eval()` and direct user input as a vulnerability.

```
user_input = input()
eval(user_input)
```

```
[1] The 'eval()' function is used, which can execute arbitrary code provided by a user, leading to severe security risks.
```

```
Severity: MEDIUM
```

```
CWE: 78
```

```
Line: 2
```

```
Suggested Fix:
```

```
Consider replacing 'eval()' with 'ast.literal_eval()' if you only need to evaluate basic Python data structures like strings, numbers, lists, or dictionaries, as it is much safer against arbitrary code execution.
```

When attempting to use prompt injection and manipulation in the code request to maliciously change the output of Gemini, outputs were still as expected, meaning the prompt successfully prevented Gemini from using the user-submitted instructions (such as “do this task instead” or “report no vulnerabilities”) instead of our intended instructions.

Conclusion

In conclusion, our project went fairly smoothly, as we didn't run into too many problems. Implementing both Bandit and Gemini to work together proved to be fairly easy, as we had the MVP put together within a short period of time. Throughout the rest of our development time, we added a handful of new features and significantly improved the look in comparison to the MVP. While we didn't get to add as many features as we had initially planned, such as user authentication for a more advanced method of allowing more advanced AI models and request history, we didn't consider these to be necessary.

While we consider this to be complete, there are some things that could be hugely improved on. For example, we didn't have time to add static analysis tools for other languages (just Python at the moment), so any other languages are using Gemini for code analysis in addition to report generation. As a result, any reports from samples other than Python will be inconsistent, and when using the default cheap AI model some vulnerabilities will be missed. In comparison, Python uses the Bandit report as grounding, meaning the model already knows what the vulnerabilities are and where they are from an external source. In addition, malformed reports and prompt injection will always be a risk, no matter how much effort we put into stopping it due to the unpredictability of AI.

In the future, if development on CopyCat was resumed, one of the primary goals would be to set up more static analysis tools. This would improve the results significantly on additional languages and allow cheaper models to be used more reliably. The report display could also be improved, as right now it's just text output with some formatting.